

GPUMODE Lecture Study Notes: Lecture 2 Ch1–3 PMPP Book

Contents

1 Core Problem the Lecture Addresses

This lecture recaps the first three chapters of *Programming Massively Parallel Processors* through the lens of CUDA programming. The central problem is how to move from sequential CPU-style computation to massively parallel GPU-style computation without losing correctness, performance intuition, or control over hardware behavior.

The lecture focuses on three foundational questions:

1. Why did computing shift from faster sequential processors toward parallel processors?
2. How does CUDA express data-parallel work across many GPU threads?
3. How do multidimensional data structures such as vectors, images, and matrices map onto CUDA grids, blocks, and threads?

The motivating bottleneck is that modern workloads require enormous throughput. Examples include graphics, weather prediction, robotics, protein folding, financial simulation, self-driving systems, and deep learning. These workloads often involve processing large arrays, tensors, images, or matrices. GPUs are effective because they provide much higher aggregate floating-point throughput than CPUs, but this throughput is only accessible when the programmer exposes enough parallelism.

The key mental shift is from asking, “How do I execute this loop faster?” to asking, “How do I replace this loop with many independent threads, each responsible for a well-defined piece of the output?”

The lecture repeatedly emphasizes that parallel programming is not merely about launching many threads. It is about writing programs where each thread knows which data it owns, where memory movement is controlled, and where synchronization is used only when needed. A naive GPU port can be correct but slow if it spends too much time transferring data, reading memory inefficiently, or launching kernels that do too little useful work.

2 Required Background

2.1 Sequential Execution Versus Parallel Execution

Traditional CPU programs are often written as sequential control flows. A simple loop such as vector addition might look like this:

```
for (int i = 0; i < n; ++i) {  
    z[i] = x[i] + y[i];  
}
```

This code expresses a sequence of operations, but the mathematical work is independent across elements. The computation for $z[0]$ does not depend on $z[1]$, and so on. A GPU implementation should exploit this independence by assigning one or more output elements to each thread.

For a vector addition of length n , the mathematical operation is

$$z_i = x_i + y_i, \quad 0 \leq i < n.$$

Each z_i can be computed independently. This is the basic structure of **data parallelism**.

2.2 Why Clock Speed Scaling Stopped Being Enough

Historically, software became faster because CPUs improved through higher clock rates, better microarchitecture, and faster memory systems. Over time, increasing clock frequency became increasingly difficult due to power and heat constraints. A simplified dynamic power model is

$$P_{\text{dynamic}} \approx CV^2f,$$

where C is effective switched capacitance, V is voltage, and f is clock frequency. Increasing frequency and voltage raises power consumption and heat dissipation. Eventually, simply making one core faster became inefficient.

The architectural response was to add more cores and more parallel execution resources. CPUs became multicore, and GPUs became widely used for general-purpose computation. However, this also shifted responsibility onto programmers and compiler/runtime systems to expose parallelism.

2.3 CPU Threads Versus GPU Threads

CPU threads are relatively heavyweight. A CPU thread has state such as a program counter, stack, register state, and operating-system scheduling metadata. Context switching between CPU threads involves saving and restoring state.

GPU threads are much lighter in the programming model. CUDA encourages launching thousands or millions of threads. The hardware is designed to keep many thread states resident and to switch between warps efficiently. Therefore, launching many CUDA threads is not something the programmer should fear.

A CPU programmer often thinks, “threads are expensive.” A CUDA programmer should instead think, “threads are the basic unit of expressing parallel work.”

2.4 Host and Device Terminology

CUDA uses a host-device model:

- **Host:** the CPU and its memory.
- **Device:** the GPU and its memory.
- **Host code:** code that runs on the CPU.
- **Device code:** code that runs on the GPU.
- **Kernel:** a special GPU function launched from host code and executed by many CUDA threads.

A typical CUDA program uses both host and device code. The host allocates memory, transfers data to the device, launches kernels, retrieves results, and frees memory.

3 Main Concepts

3.1 The Motivation for GPUs

The lecture begins with the motivation that GPUs are now central to many scientific and machine-learning workloads. The reason is not merely that GPUs are “fast” in a vague sense. The deeper

reason is that GPUs offer large throughput for workloads that can be expressed as many independent arithmetic operations over large data structures.

Modern deep learning is a strong example. Large models require enormous matrix multiplications, convolutions, attention operations, reductions, and normalization operations. These operations are mostly tensor computations. A tensor computation often contains millions or billions of independent or semi-independent arithmetic operations.

The GPU is designed to trade single-thread latency for aggregate throughput. A CPU core is optimized for sophisticated control flow, branch prediction, caching, and low-latency execution of a small number of threads. A GPU is optimized to execute a huge number of simple operations in parallel.

3.2 Data Parallelism

The main paradigm introduced in the lecture is **data parallelism**. A computation is data parallel when the same operation can be applied independently to many elements.

Examples include:

- Vector addition.
- RGB-to-grayscale conversion.
- Image blur filters.
- Matrix multiplication, at least at the level of assigning one output element per thread.

For vector addition,

$$z_i = x_i + y_i.$$

For RGB-to-grayscale conversion, each output pixel is computed from one input pixel:

$$g_{r,c} = w_R R_{r,c} + w_G G_{r,c} + w_B B_{r,c},$$

where $R_{r,c}$, $G_{r,c}$, and $B_{r,c}$ are red, green, and blue channels at row r and column c . Common luminance weights are approximately

$$w_R = 0.299, \quad w_G = 0.587, \quad w_B = 0.114.$$

Every output grayscale pixel can be computed independently. This makes the operation naturally suited to a GPU.

3.3 Same Program, Multiple Data

CUDA kernels follow a single-program, multiple-data style. Every thread executes the same kernel code, but each thread uses its own built-in identifiers to determine which data it should read and write.

The idea is

same code + different thread identity $\longrightarrow$ different data element.

This is why CUDA exposes variables such as `blockIdx`, `threadIdx`, `blockDim`, and `gridDim`. The kernel code itself is shared, but each thread calculates a different index.

3.4 CUDA Thread Hierarchy

CUDA organizes threads into a two-level hierarchy:

- A **grid** contains thread blocks.
- A **thread block** contains threads.

A kernel launch creates a grid. The grid is divided into blocks, and each block contains multiple threads. Threads inside the same block can cooperate through shared memory and synchronization. Threads in different blocks generally cannot directly synchronize within a single standard kernel launch.

For a one-dimensional launch, the global index is commonly computed as

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

Here:

- `blockIdx.x` identifies the block.
- `blockDim.x` is the number of threads per block.
- `threadIdx.x` identifies the thread inside the block.
- i is the global element index assigned to that thread.

The lecture uses the analogy that `blockIdx` is like an area code and `threadIdx` is like a local phone number. The local number alone is not globally unique; it becomes globally meaningful when combined with the block number.

3.5 Kernel Launch Syntax

CUDA launches kernels using special triple-angle-bracket syntax:

```
kernel_name<<<numBlocks, threadsPerBlock>>>(arguments);
```

The launch configuration specifies how many blocks are in the grid and how many threads are in each block. Additional optional parameters can specify dynamic shared memory size and CUDA streams, but the early examples use only the first two parameters.

For a vector of length n and a block size B , the number of blocks is usually computed using ceiling division:

$$\text{numBlocks} = \left\lceil \frac{n}{B} \right\rceil = \frac{n + B - 1}{B} \quad \text{using integer division.}$$

In C or C++, this is commonly written as

```
int threadsPerBlock = 256;
int numBlocks = (n + threadsPerBlock - 1) / threadsPerBlock;
```

This may launch more threads than there are elements. Therefore, kernels usually include a bounds check.

3.6 Bounds Checks

Because the grid size is often rounded up, some threads may correspond to indices outside the valid data range. A bounds check prevents illegal memory access:

```
if (i < n) {  
    z[i] = x[i] + y[i];  
}
```

Without this check, threads with $i \geq n$ could read or write out of bounds. That can cause incorrect results, memory corruption, or kernel failure.

3.7 Memory Allocation and Transfer

CUDA programs explicitly manage memory between host and device. The basic device-memory functions are:

- `cudaMalloc`: allocate memory on the GPU.
- `cudaFree`: free GPU memory.
- `cudaMemcpy`: copy data between host and device.

The canonical pattern is:

1. Allocate host memory.
2. Allocate device memory.
3. Copy inputs from host to device.
4. Launch GPU kernel.
5. Copy outputs from device to host.
6. Free device memory.

For toy examples this is acceptable. For high-performance applications, repeatedly moving data between CPU and GPU can dominate runtime. A better strategy is to keep data on the GPU across many operations.

3.8 CUDA Function Qualifiers

CUDA extends C/C++ with function qualifiers:

- `__global__`: declares a kernel callable from host code and executed on the device.
- `__device__`: declares a function callable from device code and executed on the device.
- `__host__`: declares a function callable from host code and executed on the host. This is the default for ordinary C/C++ functions.

A function can be both `__host__` and `__device__`, in which case the compiler emits versions for both CPU and GPU execution.

3.9 Compilation Through NVCC and PTX

CUDA code is compiled using NVIDIA's CUDA compiler driver, `nvcc`. Device code is typically compiled into PTX, which stands for Parallel Thread Execution. PTX is a low-level virtual instruction set. At runtime or installation time, PTX can be just-in-time compiled by the NVIDIA driver into GPU-specific machine code.

The conceptual pipeline is

$$\text{CUDA C/C++} \longrightarrow \text{PTX} \longrightarrow \text{GPU-specific binary instructions.}$$

This separation allows CUDA programs to target a virtual GPU instruction set while still generating device-specific code later.

4 Hardware-Level Explanation

4.1 GPU Throughput Orientation

A CPU is usually optimized for low latency on a small number of complex threads. A GPU is optimized for high throughput across many simple threads. This means that an individual GPU thread may not be fast in isolation, but the aggregate throughput across many threads can be enormous.

In a data-parallel kernel, the GPU can schedule many warps across streaming multiprocessors. A **warp** is a group of usually 32 CUDA threads that execute together in SIMT fashion. The lecture does not deeply focus on warps yet, but the thread-grid model introduced here is the software abstraction that eventually maps onto warps, SMs, and memory transactions.

4.2 Streaming Multiprocessors and Thread Blocks

A CUDA thread block is assigned to one streaming multiprocessor, or SM, for execution. Multiple blocks can reside on the same SM if resources allow. Resources include registers, shared memory, maximum resident warps, and maximum resident blocks.

Threads in a block can cooperate because they are colocated on the same SM and can access the same shared memory. This becomes crucial for optimized matrix multiplication and many other tiled algorithms.

The early lecture examples do not use shared memory, but the thread-block abstraction already anticipates it. The reason CUDA groups threads into blocks is not arbitrary. Blocks are the unit of cooperative execution and resource allocation.

4.3 Asynchronous Kernel Launch

CUDA kernel launches are asynchronous with respect to the host by default. When the CPU launches a kernel, it can continue executing other work unless it explicitly synchronizes or performs a blocking operation such as copying data back from the device.

Conceptually,

$$\text{CPU launches kernel} \not\Rightarrow \text{CPU waits for kernel completion.}$$

This has practical consequences:

- Timing a kernel using ordinary CPU timers requires synchronization.

- Errors from kernel execution may appear later unless checked carefully.
- CPU and GPU work can overlap when managed properly.

A robust program often uses error checks after CUDA API calls and after kernel launches.

4.4 Global Memory

The early examples use device global memory. Global memory is large but has high latency compared with registers and shared memory. Efficient CUDA programs need to manage global memory traffic carefully.

For vector addition, each element requires:

$$\text{read } x_i + \text{read } y_i + \text{write } z_i.$$

For single-precision floating point values, each value is 4 bytes. Therefore, each output element performs

$$2 \text{ FLOPs?}$$

Actually, vector addition performs one floating-point add per element:

$$\text{FLOPs per element} = 1.$$

The memory traffic per element is approximately

$$\text{Bytes per element} = 4 + 4 + 4 = 12 \text{ bytes},$$

ignoring caching and write-allocation details. The arithmetic intensity is therefore

$$\text{Arithmetic Intensity} = \frac{1 \text{ FLOP}}{12 \text{ bytes}} \approx 0.083 \text{ FLOPs/byte}.$$

This is extremely low. Vector addition is usually memory-bandwidth bound, not compute-bound.

4.5 Memory Coalescing Intuition

Although the lecture only hints at deeper performance issues, the indexing examples are the beginning of memory-coalescing intuition. If adjacent threads access adjacent memory locations, the GPU can combine memory accesses into efficient transactions.

For a one-dimensional vector addition, adjacent threads usually access

$$x_i, x_{i+1}, x_{i+2}, \dots$$

and similarly for y and z . This is a favorable pattern.

For multidimensional data, the mapping from thread indices to memory addresses determines whether access is coalesced. Row-major layout makes consecutive columns in the same row adjacent in memory. Therefore, mapping `threadIdx.x` to columns usually helps coalescing.

4.6 Thread Scheduling and Ordering

CUDA threads do not execute in a global sequential order. A higher thread index does not mean that the thread runs after a lower thread index. Unless synchronization is used, the programmer should not rely on ordering among threads.

This matters for correctness. A kernel is easiest to write when each thread writes a unique output element and does not depend on writes from other threads. If dependencies exist, synchronization or a different algorithmic structure is required.

5 Mathematical Reasoning

5.1 One-Dimensional Indexing

For a one-dimensional grid and one-dimensional blocks, the global thread index is

$$i = b_x B_x + t_x,$$

where

$$b_x = \text{blockIdx.x}, \quad B_x = \text{blockDim.x}, \quad t_x = \text{threadIdx.x}.$$

The valid range is

$$0 \leq i < n.$$

If n is not divisible by B_x , the grid is rounded up:

$$G_x = \left\lceil \frac{n}{B_x} \right\rceil.$$

The total number of launched threads is

$$T_{\text{launched}} = G_x B_x.$$

The number of inactive or masked-off threads due to bounds checking is

$$T_{\text{inactive}} = G_x B_x - n.$$

For example, if $n = 8000$ and $B_x = 1024$, then

$$G_x = \left\lceil \frac{8000}{1024} \right\rceil = 8.$$

The total launched threads are

$$T_{\text{launched}} = 8 \cdot 1024 = 8192.$$

The inactive threads are

$$T_{\text{inactive}} = 8192 - 8000 = 192.$$

5.2 Two Elements Per Thread

Sometimes each thread processes more than one element. If each thread processes two adjacent or logically related elements, a common indexing pattern is

$$i_0 = 2(b_x B_x + t_x), \quad i_1 = i_0 + 1.$$

Each thread handles i_0 and i_1 if they are in bounds.

Another pattern described in the lecture exercises splits a block's work into two consecutive sections. Each block processes $2B_x$ elements. The first section index is

$$i_0 = 2b_x B_x + t_x.$$

The second section index is

$$i_1 = 2b_x B_x + B_x + t_x.$$

This is different from simply assigning adjacent pairs to each thread. The distinction matters because it changes the memory access pattern and how threads within a block cover consecutive elements.

5.3 Two-Dimensional Indexing

For a two-dimensional grid and two-dimensional blocks, a thread can compute row and column indices as

$$\text{row} = \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y},$$

$$\text{col} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

This is natural for images and matrices. The bounds check is

$$0 \leq \text{row} < H, \quad 0 \leq \text{col} < W.$$

5.4 Row-Major Flattening

CUDA global memory is a flat one-dimensional address space. Multidimensional arrays are logical views over one-dimensional memory. For a row-major matrix A with height H and width W , the element at row r and column c is stored at linear index

$$\text{idx} = rW + c.$$

Thus,

$$A[r, c] = A_{\text{flat}}[rW + c].$$

The row stride is W , and the column stride is 1. Moving one row down changes the address by W elements. Moving one column right changes the address by 1 element.

For a three-dimensional image with channels stored in interleaved layout, such as height-width-channel order, the flattening can be

$$\text{idx} = (rW + c)C + ch,$$

where C is the number of channels and ch is the channel index.
 For channel-first layout, such as channel-height-width order, the flattening is

$$\text{idx} = chHW + rW + c.$$

Correct CUDA indexing requires knowing which memory layout is being used.

5.5 Image Blur Formula

The image blur example computes a mean filter. For each output pixel at row r and column c , with radius R , the output is the average over neighboring valid pixels:

$$O_{r,c} = \frac{1}{N_{r,c}} \sum_{\Delta r=-R}^R \sum_{\Delta c=-R}^R I_{r+\Delta r, c+\Delta c} \cdot \mathbf{1}[0 \leq r + \Delta r < H \wedge 0 \leq c + \Delta c < W].$$

Here $N_{r,c}$ is the number of valid pixels included in the average:

$$N_{r,c} = \sum_{\Delta r=-R}^R \sum_{\Delta c=-R}^R \mathbf{1}[0 \leq r + \Delta r < H \wedge 0 \leq c + \Delta c < W].$$

Interior pixels have

$$N_{r,c} = (2R + 1)^2.$$

Boundary pixels have fewer valid neighbors. For example, with $R = 1$, an interior pixel averages over 9 values, an edge pixel may average over 6, and a corner pixel may average over 4.

5.6 Matrix Multiplication Formula

Matrix multiplication computes

$$P = MN.$$

For square matrices of size $N \times N$,

$$P_{r,c} = \sum_{k=0}^{N-1} M_{r,k} N_{k,c}.$$

Each output element is an inner product between row r of M and column c of N .

A naive CUDA mapping assigns one thread to one output element $P_{r,c}$. That thread performs N multiply-add operations.

The total operation count for square matrix multiplication is approximately

$$\text{FLOPs} = 2N^3,$$

because each multiply-add contributes one multiplication and one addition.

The naive per-output thread performs

$$N \text{ multiplications} + (N - 1) \text{ additions} \approx 2N \text{ FLOPs.}$$

5.7 Arithmetic Intensity of Naive Matrix Multiplication

A naive one-thread-per-output matrix multiplication reads N elements from M and N elements from N for each output element, then writes one output value. For FP32, approximate memory traffic per output is

$$\text{Bytes per output} \approx 4N + 4N + 4 = 8N + 4.$$

The FLOPs per output are approximately

$$\text{FLOPs per output} \approx 2N.$$

Therefore, a rough per-thread arithmetic intensity is

$$\text{AI}_{\text{naive}} = \frac{2N}{8N + 4} \approx \frac{1}{4} \text{ FLOPs/byte} \quad \text{for large } N.$$

This ignores cache reuse. In practice, neighboring output elements reuse rows or columns, and optimized kernels intentionally exploit that reuse through tiling and shared memory. The lecture presents matrix multiplication only as an introductory example, but it foreshadows the need for more advanced GPU optimization.

6 Code Walkthrough

6.1 Vector Addition Kernel

A minimal CUDA vector addition kernel is:

```
__global__ void vecAddKernel(  
    const float* x,  
    const float* y,  
    float* z,  
    int n  
) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (i < n) {  
        z[i] = x[i] + y[i];  
    }  
}
```

6.1.1 Line-by-Line Explanation

- `__global__` marks the function as a CUDA kernel. It is launched from host code and executed on the GPU.
- `x`, `y`, and `z` are pointers to device memory.
- `n` is the number of vector elements.
- `blockIdx.x * blockDim.x + threadIdx.x` computes the global one-dimensional thread index.
- The bounds check ensures that extra rounded-up threads do not access invalid memory.

- Each valid thread reads one element from x , one element from y , adds them, and writes one element to z .

6.1.2 Hardware Behavior

Adjacent threads access adjacent elements. For example, consecutive threads read $x[i]$, $x[i+1]$, $x[i+2]$, and so on. This is favorable for coalesced global memory access. However, the arithmetic intensity is very low, so the kernel is usually limited by memory bandwidth.

6.2 Host Code for Vector Addition

The host-side flow is:

```
void vecAddHost (
    const float* x_h,
    const float* y_h,
    float* z_h,
    int n
) {
    size_t bytes = n * sizeof(float);

    float* x_d = nullptr;
    float* y_d = nullptr;
    float* z_d = nullptr;

    cudaMalloc((void**)&x_d, bytes);
    cudaMalloc((void**)&y_d, bytes);
    cudaMalloc((void**)&z_d, bytes);

    cudaMemcpy(x_d, x_h, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(y_d, y_h, bytes, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int numBlocks = (n + threadsPerBlock - 1) / threadsPerBlock;

    vecAddKernel<<<numBlocks, threadsPerBlock>>>(x_d, y_d, z_d, n);

    cudaMemcpy(z_h, z_d, bytes, cudaMemcpyDeviceToHost);

    cudaFree(x_d);
    cudaFree(y_d);
    cudaFree(z_d);
}
```

6.2.1 Major Steps

1. Compute the number of bytes required for the vector.
2. Allocate three arrays in GPU global memory.
3. Copy input arrays from CPU memory to GPU memory.
4. Compute the launch configuration.

5. Launch the kernel.
6. Copy the result back to CPU memory.
7. Free GPU memory.

6.2.2 Performance Implication

For a single vector addition, this full process is inefficient because the cost of copying data to and from the GPU can dominate the cost of the addition itself. The GPU becomes valuable when the data remains on the device across many operations.

6.3 Error Checking Helper

The lecture notes that error checking is important in real CUDA programs. A simple helper is:

```
#define CUDA_CHECK(call) \
    do { \
        cudaError_t err = (call); \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, \
                cudaGetErrorString(err)); \
            exit(EXIT_FAILURE); \
        } \
    } while (0)
```

This can be used as follows:

```
CUDA_CHECK(cudaMalloc((void**)&x_d, bytes));
CUDA_CHECK(cudaMemcpy(x_d, x_h, bytes, cudaMemcpyHostToDevice));

vecAddKernel<<<numBlocks, threadsPerBlock>>>(x_d, y_d, z_d, n);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

6.3.1 Why This Matters

CUDA errors can otherwise be silent or delayed. Since kernel launches are asynchronous, a kernel error might not appear at the launch line unless explicitly checked. Calling `cudaGetLastError` catches launch errors, while `cudaDeviceSynchronize` catches execution errors.

6.4 RGB-to-Grayscale Kernel

A representative grayscale conversion kernel is:

```
__global__ void rgbToGrayscaleKernel(
    const unsigned char* rgb,
    unsigned char* gray,
    int width,
    int height
) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
```

```

if (row < height && col < width) {
    int grayOffset = row * width + col;
    int rgbOffset = 3 * grayOffset;

    unsigned char r = rgb[rgbOffset + 0];
    unsigned char g = rgb[rgbOffset + 1];
    unsigned char b = rgb[rgbOffset + 2];

    gray[grayOffset] =
        static_cast<unsigned char>(
            0.299f * r + 0.587f * g + 0.114f * b
        );
}
}

```

6.4.1 Indexing Logic

The output grayscale image has shape $H \times W$. The flattened grayscale index is

$$\text{grayOffset} = \text{row} \cdot W + \text{col}.$$

For an interleaved RGB image, each pixel has three consecutive channel values, so the RGB offset is

$$\text{rgbOffset} = 3(\text{row} \cdot W + \text{col}).$$

The thread reads red, green, and blue values, computes a weighted sum, and writes one grayscale value.

6.4.2 Hardware Behavior

This kernel maps one thread to one output pixel. If `threadIdx.x` maps to adjacent columns, neighboring threads read adjacent RGB triplets and write adjacent grayscale outputs. This is a natural memory access pattern for row-major images.

6.5 Image Mean Filter Kernel

The lecture discusses an image blur kernel using a mean filter. A representative CUDA kernel is:

```

__global__ void meanFilterKernel(
    const float* input,
    float* output,
    int width,
    int height,
    int radius
) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (row >= height || col >= width) {
        return;
    }
}

```

```

float sum = 0.0f;
int count = 0;

for (int dy = -radius; dy <= radius; ++dy) {
    for (int dx = -radius; dx <= radius; ++dx) {
        int rr = row + dy;
        int cc = col + dx;

        if (rr >= 0 && rr < height && cc >= 0 && cc < width) {
            int idx = rr * width + cc;
            sum += input[idx];
            count += 1;
        }
    }
}

int outIdx = row * width + col;
output[outIdx] = sum / static_cast<float>(count);
}

```

6.5.1 Line-by-Line Explanation

- The thread computes its output pixel coordinates using two-dimensional CUDA indexing.
- Threads outside the image return immediately.
- The nested loops iterate over a square neighborhood around the output pixel.
- Boundary checks prevent out-of-bounds reads at image edges and corners.
- `count` tracks how many valid pixels were included.
- The output is the average of all valid neighboring pixels.

6.5.2 Hardware Behavior

Each thread writes one output pixel but reads multiple input pixels. For radius R , an interior output pixel reads

$$(2R + 1)^2$$

input values. This increases arithmetic and memory work per thread. Neighboring threads read overlapping neighborhoods, which creates data reuse. A later optimized version could load tiles into shared memory so that neighboring threads reuse data explicitly instead of redundantly loading from global memory.

6.5.3 Visual Concept

Draw a two-dimensional image grid. Highlight one output pixel in the center, then draw a square neighborhood of radius R around it. Show that corner pixels have a truncated neighborhood because some positions fall outside the image.

6.6 Matrix Multiplication Kernel

A naive square matrix multiplication kernel is:

```
__global__ void matMulKernel(  
    const float* M,  
    const float* N,  
    float* P,  
    int width  
) {  
    int col = blockIdx.x * blockDim.x + threadIdx.x;  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
  
    if (row < width && col < width) {  
        float acc = 0.0f;  
  
        for (int k = 0; k < width; ++k) {  
            acc += M[row * width + k] * N[k * width + col];  
        }  
  
        P[row * width + col] = acc;  
    }  
}
```

6.6.1 Indexing Logic

For output element $P_{r,c}$, the thread computes

$$P_{r,c} = \sum_{k=0}^{W-1} M_{r,k} N_{k,c}.$$

The row-major linear addresses are

$$M_{r,k} \mapsto rW + k,$$

$$N_{k,c} \mapsto kW + c,$$

$$P_{r,c} \mapsto rW + c.$$

6.6.2 Hardware Behavior

Each thread computes one output element. Threads in the same row of P reuse the same row of M , and threads in the same column of P reuse the same column of N . However, the naive kernel does not explicitly stage this data in shared memory. It relies only on caches and natural locality.

The access to $M[\text{row} * \text{width} + k]$ is identical for threads computing different columns in the same row. The access to $N[k * \text{width} + \text{col}]$ varies by column, so adjacent threads may access adjacent values of N for a fixed k . This can be favorable for coalescing depending on the mapping of threads to columns.

6.6.3 Visual Concept

Draw matrix M on the left, matrix N on the top or right, and output matrix P . Highlight one row of M and one column of N meeting at one output element $P_{r,c}$. Then show a two-dimensional CUDA block covering a tile of output elements.

7 Performance Intuition

7.1 Correct Parallelism Is Not the Same as High Performance

A central theme is that making a program parallel is only the first step. A GPU program can be correct yet slow. Common reasons include:

- Too much host-device data transfer.
- Poor memory access patterns.
- Low arithmetic intensity.
- Uncoalesced global memory access.
- Too little work per kernel launch.
- Excess synchronization.
- Data-dependent irregularity.

7.2 Vector Addition Is Memory-Bound

Vector addition performs one arithmetic operation per output but moves multiple values through memory. Its arithmetic intensity is approximately

$$\frac{1}{12} \text{ FLOPs/byte}$$

for FP32. This is far too low to saturate the GPU's compute units. The limiting factor is memory bandwidth.

The roofline-style upper bound is

$$\text{Attainable FLOPs/s} \leq \min(\text{Peak FLOPs/s}, \text{Bandwidth} \cdot \text{Arithmetic Intensity}).$$

For low arithmetic intensity, the bandwidth term dominates.

7.3 Host-Device Transfers Can Dominate

For small examples, the full process of copying data to the GPU, launching one kernel, and copying data back is often slower than simply using the CPU. The GPU is most effective when many computations are performed after data is already on the device.

A poor workflow is

$$\text{copy to GPU} \rightarrow \text{one tiny kernel} \rightarrow \text{copy to CPU}.$$

A better workflow is

copy to GPU $\rightarrow$ many kernels $\rightarrow$ copy final result to CPU.

Deep learning frameworks follow this second pattern. Model parameters, activations, gradients, and optimizer states usually remain on the GPU across many operations.

7.4 One Thread Per Output Element

The lecture repeatedly uses the strategy

one thread $\longleftrightarrow$ one output element.

This is simple and often correct for introductory kernels. It works well when each output element can be computed independently.

Examples:

- One thread computes one vector element.
- One thread computes one grayscale pixel.
- One thread computes one blurred pixel.
- One thread computes one matrix output element.

However, optimized kernels may assign multiple elements to each thread, multiple threads to one output tile, or one warp to a structured group of outputs. The one-thread-per-output model is a starting point, not the final form of high-performance CUDA.

7.5 Block Size Choice

The lecture uses examples such as 256 threads per block and notes the maximum of 1024 threads per block. The block size affects occupancy, scheduling, memory behavior, and resource usage.

A block size should generally be a multiple of warp size:

$$B = m \cdot 32,$$

where m is an integer. Common introductory choices are 128, 256, or 512 threads per block.

Too few threads per block may underutilize hardware. Too many threads per block may reduce flexibility or increase resource pressure. Later optimization requires considering registers per thread, shared memory per block, occupancy, and memory access patterns.

7.6 Data Shape Affects Performance

The lecture notes that performance can vary depending on data shape. In deep learning, sequence length, batch size, tensor layout, and matrix dimensions can change which kernel is optimal.

For example, LLM inference with batch size 1 and token-by-token generation may fail to fully utilize the GPU. The GPU has enormous throughput, but a small batch may not expose enough parallel work or arithmetic intensity. Batching requests can increase utilization, but it also introduces latency and scheduling trade-offs.

7.7 Synchronization and Dependencies

Not all computations are embarrassingly parallel. If output elements depend on each other, the algorithm may require synchronization or a different parallel formulation.

Prefix sum is a classic example. The sequential form is simple:

$$y_i = \sum_{j=0}^i x_j.$$

But each output depends on previous inputs. A high-performance parallel prefix sum requires a nontrivial tree-like algorithm. This illustrates why parallel algorithm design can be less intuitive than sequential algorithm design.

7.8 Matrix Multiplication Foreshadows Tiling

The naive matrix multiplication kernel is useful pedagogically, but high-performance GEMM requires tiling. The key reason is data reuse.

For a tile of P , a block of threads can reuse a tile of M and a tile of N . Instead of each thread loading all values independently from global memory, a cooperative block can load tiles into shared memory:

$$M_{\text{tile}} \subset M, \quad N_{\text{tile}} \subset N.$$

Then many multiply-adds reuse the staged data. This raises arithmetic intensity and reduces global memory traffic.

8 Common Misconceptions

8.1 Misconception: CUDA Threads Are Like CPU Threads

CUDA threads are much lighter than CPU threads in the programming model. GPU programs commonly launch thousands or millions of threads. The overhead model is different from CPU multithreading.

8.2 Misconception: A Higher Thread Index Runs Later

Thread indices identify work; they do not define global execution order. A thread with a larger `threadIdx.x` or global index is not guaranteed to run after a smaller one.

8.3 Misconception: `threadIdx.x` Is Globally Unique

`threadIdx.x` is local to a block. Every block has threads with `threadIdx.x` values starting at 0. To compute a globally unique one-dimensional index, combine `blockIdx.x`, `blockDim.x`, and `threadIdx.x`.

8.4 Misconception: If the Kernel Is Parallel, It Must Be Fast

Parallelism is necessary but not sufficient. Performance also depends on memory bandwidth, access coalescing, arithmetic intensity, occupancy, synchronization, data transfer overhead, and hardware utilization.

8.5 Misconception: The Grid Must Exactly Match the Data Size

The grid often intentionally launches more threads than needed using ceiling division. Extra threads are made harmless with bounds checks.

8.6 Misconception: Multidimensional Arrays Are Stored Multidimensionally

Memory is linear. A matrix, image, or tensor is a logical interpretation over flat memory. Correct indexing requires understanding layout and strides.

8.7 Misconception: Matrix Multiplication Is Just a Simple Nested Loop

Mathematically, matrix multiplication is simple. On GPUs, high-performance matrix multiplication is one of the deepest optimization topics because it involves tiling, shared memory, register blocking, warp-level computation, tensor cores, memory coalescing, and scheduling.

9 Practical Takeaways

1. Think in terms of output ownership. First ask which thread owns which output element.
2. Use CUDA built-in variables to map blocks and threads to data indices.
3. Always include bounds checks when grid dimensions are rounded up.
4. Understand whether your data is row-major, column-major, channel-first, or channel-last.
5. Treat host-device transfer as expensive. Keep data on the GPU when possible.
6. Use error checking during development. Silent CUDA errors waste debugging time.
7. Do not assume thread execution order unless synchronization explicitly guarantees it.
8. Recognize memory-bound kernels. Low arithmetic intensity means more FLOPs will not help unless memory traffic is reduced.
9. Start with one-thread-per-output kernels, then optimize with tiling, shared memory, vectorized loads, or warp-level strategies when needed.
10. Matrix multiplication is the gateway example for understanding serious GPU performance engineering.

10 Project or Experiment Ideas

10.1 Experiment 1: Vector Addition Baseline

Implement vector addition in CPU C++, naive CUDA, and PyTorch. Measure runtime for different vector sizes:

$$n \in \{10^3, 10^4, 10^5, 10^6, 10^7, 10^8\}.$$

Separate the timing into:

- Host-to-device copy time.

- Kernel execution time.
- Device-to-host copy time.
- Total end-to-end time.

Expected lesson: kernel time may be small, but transfer overhead can dominate for small or one-off computations.

10.2 Experiment 2: Block Size Sweep

Run vector addition with block sizes

$$B \in \{32, 64, 128, 256, 512, 1024\}.$$

Measure throughput in GB/s:

$$\text{Bandwidth} = \frac{12n}{t}$$

for FP32 vector addition, where t is kernel runtime in seconds. Compare the measured bandwidth to the GPU's theoretical memory bandwidth.

10.3 Experiment 3: RGB-to-Grayscale CUDA Extension

Write an RGB-to-grayscale CUDA kernel and call it from Python using a PyTorch C++/CUDA extension. Compare against a PyTorch implementation.

Measure:

- Runtime for different image sizes.
- Correctness versus PyTorch.
- Effect of channel-last versus channel-first layout.

10.4 Experiment 4: Mean Filter With and Without Shared Memory

Implement two versions of image blur:

1. Naive version where each thread reads its neighborhood from global memory.
2. Tiled version where each block loads a tile and halo region into shared memory.

For radius R , the naive interior read count per output is

$$(2R + 1)^2.$$

The shared-memory version should reduce redundant global memory reads across neighboring threads.

10.5 Experiment 5: Naive Matrix Multiplication

Implement naive one-thread-per-output matrix multiplication. Compare against `torch.matmul` or `cuBLAS`.

Use sizes

$$N \in \{128, 256, 512, 1024, 2048\}.$$

Compute achieved GFLOPs/s:

$$\text{GFLOPs/s} = \frac{2N^3}{t \cdot 10^9}.$$

Expected lesson: naive `matmul` is much slower than `cuBLAS` because it does not fully exploit tiling, shared memory, registers, warp-level reuse, or tensor cores.

10.6 Experiment 6: Indexing Exercises

Write small kernels that only compute and store their global indices. For example, create an output array where each thread writes

$$\text{out}[i] = i.$$

Then extend to two-dimensional indexing where each thread writes

$$\text{out}[r, c] = rW + c.$$

This is a simple way to verify that the grid, block, and flattening logic are correct.

10.7 Experiment 7: Bounds Check Failure Demo

Intentionally remove the bounds check from vector addition and run with a vector size not divisible by block size. Observe incorrect behavior or runtime errors. Then restore the bounds check and confirm correctness.

This experiment teaches why the condition

$$i < n$$

is not optional.

11 Final Mental Model

CUDA programming begins by replacing sequential loops with a grid of lightweight GPU threads. Each thread runs the same kernel code, but built-in identifiers such as `blockIdx`, `threadIdx`, and `blockDim` let it compute a unique data index. The programmer's first job is to map threads to output elements correctly. The second job is to make that mapping efficient for the GPU memory hierarchy.

A CUDA kernel is not just a function. It is a recipe executed by a massive grid of threads. Performance comes from giving those threads independent work, arranging their memory accesses predictably, minimizing unnecessary data movement, and exploiting the hardware hierarchy from global memory down to shared memory and registers.

The first three PMPP chapters establish the foundation: GPUs exist because throughput scaling replaced clock-speed scaling; CUDA expresses heterogeneous host-device computation; data parallelism maps naturally to grids and blocks; and multidimensional arrays are ultimately flat memory with indexing rules. Vector addition, grayscale conversion, image blur, and matrix multiplication are not merely toy examples. They are the first instances of the same core GPU programming pattern: assign ownership, compute indices, guard bounds, access memory carefully, and reason about throughput.